WAB

Comparative Evaluation of the XGBoost Boosters gbtree and gblinear

In Terms of Predictive Accuracy and Efficiency for Binary User Classification

For the Course Algorithms and Data Structures

submitted to the Department of
Computer Science and Business Informatics / Economics
of
Provadis School of International Management and Technology
by

Jeremy Blum
D960
Informatik BIN-T24-F3
jeremy.blum@stud-provadis-hochschule.de

First Reviewer: Abdullah Lamya
Submission Deadline: 07.10.2025

Table of Contents

List of Figures

Glossary

AUC-ROC	Area Under the Receiver Operating Characteristic Curve - a performance metric that measures the quality of predictions across all classification thresholds.
Binary Classification	A supervised learning task where the goal is to classify instances into one of two possible classes (e.g., spam vs. not spam).
Boosting	An ensemble method that sequentially combines multiple weak learners to create a strong predictive model, with each new model focusing on correcting errors of previous ones.
Cross-Validation	A statistical method for evaluating machine learning models by partitioning data into training and validation sets multiple times.
F1-Score	The harmonic mean of precision and recall, providing a balanced measure of classifier performance.
Gradient Boosting	A boosting technique that frames the ensemble learning process as gradient descent optimization in function space.
Machine Learning	The automated detection of meaningful patterns in data, enabling systems to improve performance on a specific task through experience.
Overfitting	The production of an analysis which corresponds too closely to training data and may fail to generalize to new data or predict future observations reliably.

Precision	The proportion of positive predictions that are actually correct (True Positives / (True Positives + False Positives)).
Recall	The proportion of actual positive cases that are correctly identified (True Positives / (True Positives + False Negatives)).
Supervised Learning	A machine learning paradigm where algorithms learn from labeled training data (input-output pairs) to make predictions on new, unseen data.
XGBoost	Extreme Gradient Boosting - a scalable machine learning system for tree boosting with optimized performance and regularization.

List of Tables

List of Abbreviations

BSc .... Bachelor of Science

ML .... Machine Learning

1 Introduction to Topic and Research Question

1.1 Background and Motivation

Machine Learning (ML) has evolved into a central paradigm in computer science, providing methods to automatically detect meaningful patterns in data and and make predictions based on these patterns. In contrast to traditional programmed algorithms, learning systems are capable of adapting to new data and change their behavior accordingly[1]. This data-driven approach allows computers to solve problems that are too complex to tackle with handcrafted methods, and it underpins real-world applications. For instance, *"Smart classifiers protect our email by learning from massive amounts of spam data and user feedback; advertising systems learn to match the right ads with the right context; fraud detection systems protect banks from malicious attackers; anomaly event detection systems help experimental physicists to find events that lead to new physics."*[2] - tasks that rely on extracting insights from large datasets rather than explicit programming.

Among the fundamental approaches in ML is *supervised learning*, were a model is trained on labeled examples (input-output pairs) to predict the correct output for new, unseen inputs. The learning aims to generalize from the training data and make accurate predictions on future data[3]. A common use case is **classification**, especially binary classification, where the algorithm assigns each instance to one of two possible categories (e.g. "postitive" vs. "negative") based on its features[4]. Binary classification is a basic yet critical task in ML, forming the basis for many decision making systems. Given its prevalence, improving the accuracy and efficiency of binary classification models is of high importance.

1.2 Problem Statement

In modern machine learning practice, it is common to combine multiple models to improve predictive performance. Such techniques are known as *ensemble methods*, which aggregate the outputs of several base learners to achieve better accuracy and robustness than a single model alone[5]. One successful ensemble method is **boosting**, where models are added iteratively, so that each new model can correct the errors of the ensemble built so far[6]. Gradient boosting methods have demonstrated state-of-the-art results on many classification tasks by iteratively fitting new models to the residual errors of the previous models.

XGBoost (eXtreme Gradient Boosting) is a advanced implementation of gradient boosting that introduces several optimizations to enhance both predictive accuracy and computational effi-

[1] **osisanwo2017**.
[2] **chen2016**.
[3] **kotsiantis2007**.
[4] **kotsiantis2007**.
[5] **natekin2013**.
[6] **natekin2013**.

ciency. In addition to the standard boosting procedure, XGBoost includes innovations like explicit regularazation of model complexity, parallelized tree construction, and cache-aware data structures for faster access, which makes it a highly efficient and powerful ensemble learner[7]. Within the XGBoost framework, the choice of *booster* (the type of base learner) significantly influences the model's behavior and performance. The library offers two primary booster options: **gbtree**, which uses decision trees capable of capturing complex, nonlinear relationships in the data. The linear booster **gblinear**, produces an additive linear model (with weight regularazation) analogous to logistic or linear regression[8]. In other words, `gbtree` booster relies on treebased models, while `gblinear` uses linear functions as base learners.[9].

Tree-based ensembles are generally known for their strong predictive power across diverse tasks[10], whereas linear models tend to train faster and can perform well in high-dimensional feature spaces (for example, text classification with thousands of sparse features)[11]. While XGBoost's tree booster is often observed to achieve superior accuracy on complex datasets, the linear booster may has advantages in terms of training speed and simplicity, especially when the data relationships are linear. It remains an open question how these two booster types compare head-to-head in a sprecific domain. Therefore, This study aims to comparatively evaluate **gbtree** and **gblinear** in terms of their predictive accuracy and efficiency on a binary user classification task. By analyzing their performance under the same conditions, the work seeks to identify the trade-offer between the nonlinear modeling capacity of tree ensembles and the potential speed or simplicity benefits of linear models. This investigation will inform practitioners about which booster is better suited for a given problem setting and contribute to a deeper understanding of XGBoost's capabilities.

[7]**chen2016**.
[8]**xgboost2024**.
[9]**xgboost2024**.
[10]**chen2016**.
[11]**osisanwo2017**.

2 State of Research

2.1 Supervised Learning and Binary Classification

Machine learning can be broadly defined as ‘*"the automated detection of meaningful patterns in data"*[12]. One of the most common paradigms is **supervised learning**, in which the algorithm learns from example inputs **and** their known outputs (lables) in order to predict the labels of new, unseen inputs. In a classic definition, *"supervised machine learning is the search for algorithms that reason from externally supplied instances to produce general hypotheses, which then make predictions about future instances"*[13]. The goal is thus to induce a model (the *hypothesis*) that maps input features to an output (target) based on labeled training examples, and to use this model for predicting the labels of test instances where the true labels are unknown[14]. Supervised learning encompasses both regression (predicting contiuous quantities) and **classification** (predicting discrete class labels). In particular, classification deals with cases where the output variable assumes a finite set of values[15]. Often the classes are unordered categories such as "spam" vs. "not spam" in an email filter, or a user being "churn" vs "not churn" in a marketing context. When there are exactly two possible classes, we talk about **binary classification**. Commonly, one class is designated as the "positive" class (e.g. "spam", "churn") and the other as "negative", and the classifier's task is to assign each example to one of these two labels. Binary classification is widely used and is one of *"the tasks most frequently carried out by the intelligent systems"*[16]. Examples include medical diagnoses (disease vs. no disease), fraud detection (fraudulent vs. legitimate transaction), and many others. Successful binary classifiers can be built using a variety of algorithms like decision trees, support vector machines and neural networks, as long as they can learn to differentiate between the two classes based on input features. The focus in supervised learning is on **generalization**: the learned model should not merely memorize the training data, but caputre the underlying patterns so that it predicts correctly on **new** data.

In evaluating supervised classifiers, we care both about their **predictive accuracy** (how well they correctly classify new instances) and other factores like complexity or runtime. The performance of a learning algorithm involves multiple considerations, including the **accuracy** it achieves and the **time** or computational cost required to train and use the model. In practice there can be trade-offs: a more complex model might achieve higher accuracy but at the expense of longer training time or greater resource usage. Therefore, comparative evaluations of classifiers often consider not only accuracy but also **efficiency** aspects like training speed, memory consumption and model simplicity.

[12]**shalev2014**.
[13]**kotsiantis2007**.
[14]**kotsiantis2007**.
[15]**kotsiantis2007**.
[16]**osisanwo2017**.

2.2 Boosting and Gradient Boosting Machines

Ensemble methods are techniques that combine multiple learning models to improve predictive performance. *"**Boosting** is an algorithmic paradigm that grew out of a theoretical question and became a very practical machine learning tool."*[17]. The fundamental idea of boosting is to **sequentially** add and integrate many weak models (weak learners) to create a single strong predictive model. Each weak learner may only be slightly better than random guessing, but by amplifying the accuracy of weak learners through iterative combination, boosting can achieve high overall accuracy[18]. Boosting algorithms start by training an initial model, then at each subsequent round they focus on the training examples that previous models mispredicted, training the next model to correct those errors. In this way, boosting concentrates on the "hard" cases and progressively improves performance. In contrast to bagging methods like random forests (which train many models independently and average them), boosting is a **constructive, sequential process**. For example, the popular AdaBoost algorithm assigns higher weights to misclassified instances, so that the next weak learner focuses more on those difficult cases. A boosting algorithm can in theory combine enough of the weak learners too achieve very high accuracy[19].

To place boosting on a theoretical footin, researchers recast it as an optimization problem. This gave rise to **gradient boosting**, which views boosting as a gradient descent procedure in function space. In **gradient boosting machines (GBMs)**, *"the principle idea behind this algorithm is to construct the new base-learners to be maximally correlated with the negative gradient of the loss function, associated with the whole ensemble."*[20]. in simpler terms, at each iteration the algorithm looks at where the current model is erring (according to the gardient of the loss) and trains a weak learner to address those errors. This framework is ver flexible, allowing the use of different loss functions like *squarred error* for regression, *logistic loss* for classification, etc. And boosting will adapt to minimize that loss[21]. The approach also provides some theoretical justification for why boosting often doesn't overfit easily: each new learner is reducing a well-defined objective, and regularization techniques can be applied to that objective (e.g., limiting tree depth, shrinkage of step size, etc.). Overall, boosting methods have proven to be powerful achieving resulsts in many tasks, due to its ability to *sequentially correct errors* and *refine* an ensemble model.

2.3 XGBoost

XGBoost (eXtreme Gradient Boosting) is a popular library for gradient boosting, particularly optimized for efficiency and model performance. It was introduced by Chen and Guesting

[17]**shalev2014**.
[18]**shalev2014**.
[19]**osisanwo2017**.
[20]**natekin2013**.
[21]**natekin2013**.

in 2016 and quickly became popular in the machine learning community due to its winning performance in many data science competitions and practical applications. In grdient boosting, XGBoost uses decision trees as the base learners (hence it is a gradient *tree* boosting system). What sets XGBoost apart are the various innovations and engineering optimizations that make it exceptionally fast and scalable[22]. XGBoost was a dominant tool in many Kaggle competitions, from 29 winning solutions, 17 used XGBoost as the primary model[23].

Scalability and speed: XGBoost is designed for speed. It *"runs more than ten times faster than existing popular solutions on a single machine and scales to billions of examples"*[24]. The algorithm achieves this through a combination of innovations. It exploits hardware capabilities, such as cache optimization and data compression to minimize memory access costs. XGBoost can utilize multiple CPU cores during training, and also supports distributed computing across clusters for very large data. It has out-of-core computation support, which means it can handle datasets that don't fit in memory by using disk storage[25].

Handling sparsity: Real world data often contains missing values or sparsity (e.g., one-hot encoded attributes). XGBoost introduced a "sparsity-aware algorithm" for tree splitting that efficiently handles sparse input without unnecessary computations on missing features[26]. The tree building algorithm in XGBoost can skip over missing values with a default direction for each split, and it only iterates over present feature values, making computation *linear in the number of non missing entries* rather than total entries[27]. This is particularly useful for high-dimensional data where many features may be absent.

Approximate tree learning: Another innovation is a *weighted quantile sketch* for approximate finding of split points in the data[28]. Instead of sorting all data values at each candidate split (which can be slow when there are many possible splits), XGBoost can use this sketch to efficiently propose candidate splits that approximatly minimize loss.

Regularization: A key feature is the inclusion of regularization terms to prevent overfitting. The objective function of XGBoost is the loss plus a penalization term for model complexity[29]. In particular, each tree's complexity is penalized through both the number of leaves and the magnitude of leaf weights. This means XGBoost doesn't just greedily minimize training loss - it also considers how complex each additional tree is, and it favors simpler trees that achieve a similar reduction in loss. The regularization term *"regularization term helps to smooth the final learnt weights to avoid overfitting"*[30], and encourages the model to use *"simple and predictive*

[22] **chen2016**.
[23] **chen2016**.
[24] **chen2016**.
[25] **chen2016**.
[26] **chen2016**.
[27] **chen2016**.
[28] **chen2016**.
[29] **chen2016**.
[30] **chen2016**.

functions"[31].

2.4 Booster Types: gbtree and gblinear

[31] **chen2016**.

3 Methodological Approach to the Investigation

A	B
1	2

Table 2: Example Table

4 Conducting the Investigation

5 Results of the Investigation and Discussion

6 Conclusion and Outlook

Here is an example of how to display a figure:

Figure 1: Example - Provadis Logo

Important: Tables (see Table 2) and figures (see Figure 1) must be referenced in the text!

Here is an example of a citation[32]

[32] **kotsiantis2007**.

AI Documentation

Every use of AI must be documented. For this purpose, a separate AI directory is inserted after the bibliography, which transparently shows all AI-generated content, the systems used, the "prompts" used, and the further use of the AI output.

System	**Prompt**	**Usage**
ChatGPT 1	What criteria should I use to select a leader?	further developed
ChatGPT 2	Write a text covering the following topics: Personnel marketing and its importance for a company – the connection to employer branding – the effects of personnel marketing strategy on recruiting.	modified: passages omitted
ChatGPT 3	Draft an outline for a term paper on recruiting	unchanged
Elicit 1	Which elements should be included in an Employer Branding Plan?	passages revised

If no AI was used, the directory only contains the entry: "No AI was used."

Declaration of Authorship

"I hereby confirm that I have written the present work personally and independently and have not used any sources or aids other than those indicated. All passages that have been taken literally or analogously from other sources have been identified as such. The drawings, figures and tables in this work have been created by me or have been provided with an appropriate source reference. This work has not been submitted by me to other universities in the same or

Place, Date

similar form for obtaining an academic degree."

Signature